CIS*3110 Operating Systems

Week 11

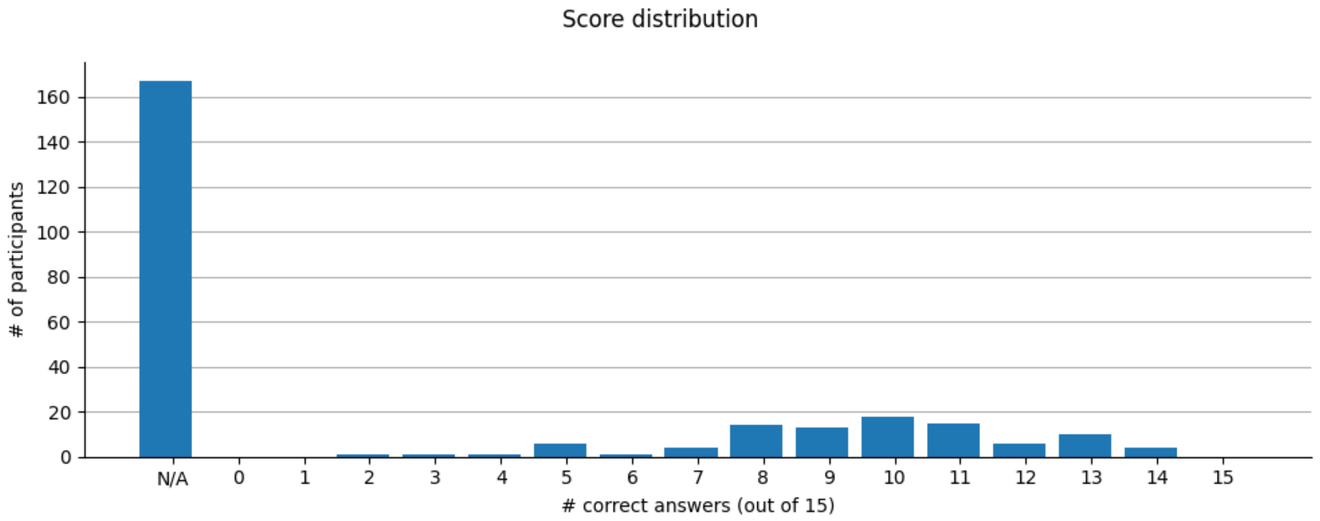
Lecture #18 File system intro

Prof. Yulia Kotseruba

March 19, 2026

Quiz 16

- **64%** average (up from 58% in quiz 15)
- 3 hardest questions:
  - (46%) In Peterson's algorithm, if flag[0]=0, flag[1]=1 and turn=0, which thread will go into CS? a) 0 b) 1 c) whoever set the flag last d) whoever set the turn last
  - (52%) Processes A=B=5s arrive at t=0. Which scheduler will give a better average turnaround time? a) RR(1) b) FIFO c) both are the same d) cannot determine
  - (54%) What is not a feature of semaphores? a) mutual exclusion b) process synchronization c) resource management d) deadlock prevention



Recap

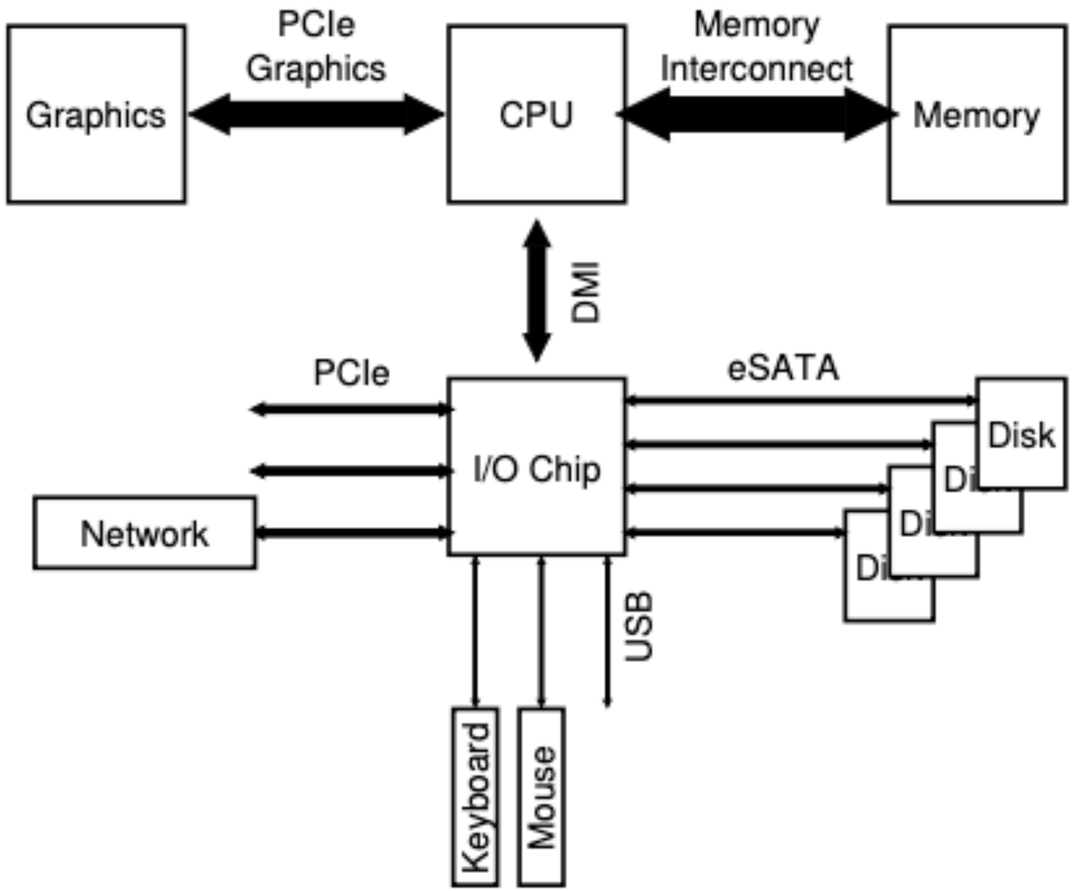
- Concurrency can cause issues that are often hard to find and fix
  - race conditions
  - livelocks
  - deadlocks
    - 4 conditions for deadlock: mutual exclusion, hold-and-wait, circular wait, no preemption
    - all 4 must hold at the same time for a deadlock to occur
    - prevent any of them and deadlocks will not happen

Recap

- Dealing with deadlocks
  - Prevention
    - Do not allow conditions for the deadlocks to arise
    - Requires rewriting the code
  - Avoidance
    - Use scheduling to schedule threads such that deadlock cannot occur
    - Don’t need to rewrite the code
  - Recovery
    - Allow deadlocks to happen, detect them, and recover

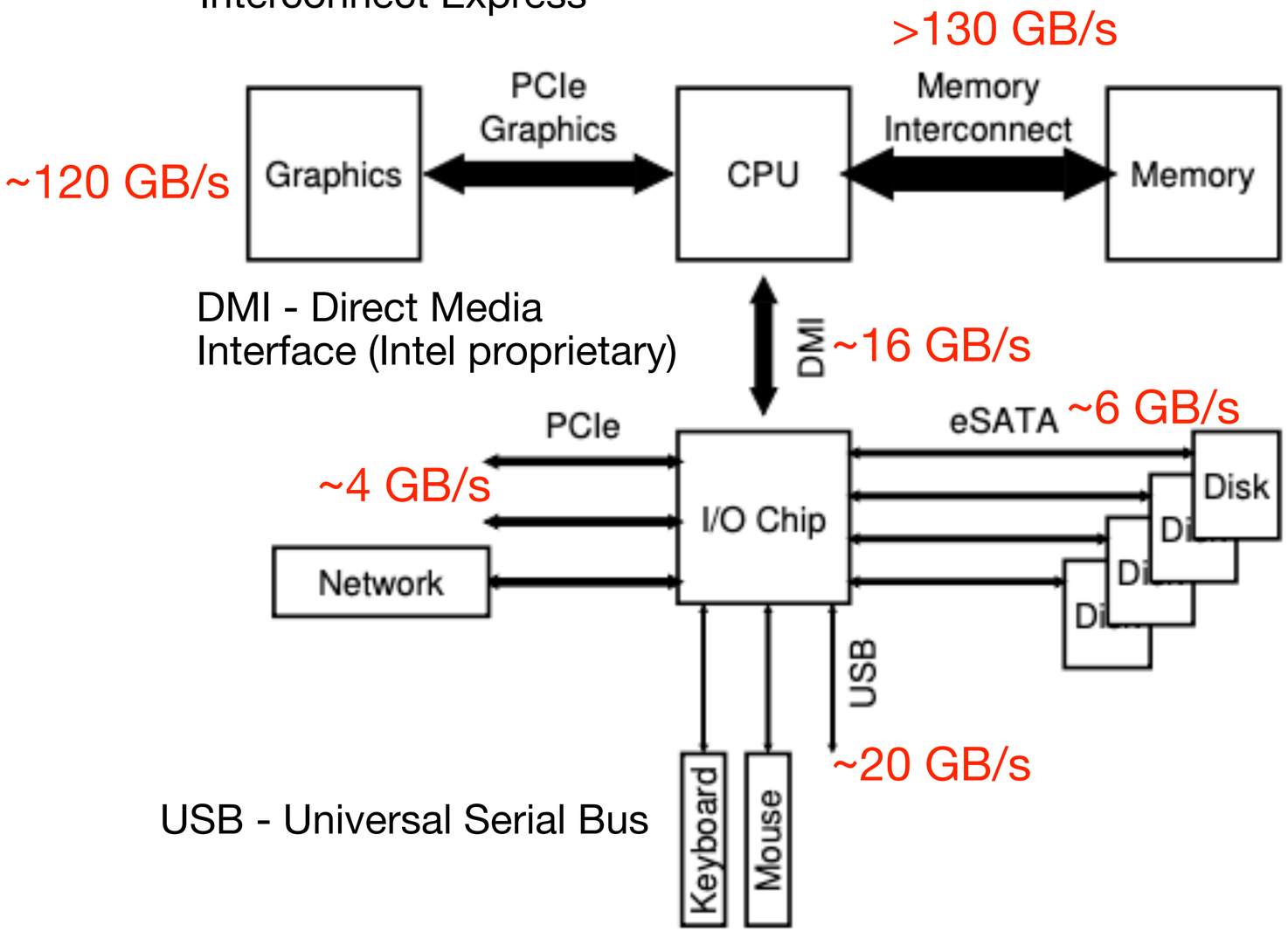
I/O devices

- Buses connect devices to CPU
- Hierarchy of buses similar to memory
  - High-throughput expensive buses
    - fewer, shorter, and closer to CPU
    - connect 1-2 devices
  - Low-throughput cheap buses
    - more, longer, and farther away from CPU
    - connect multiple devices



I/O devices

PCIe- Peripheral Component Interconnect Express



DMI - Direct Media Interface (Intel proprietary)

eSATA - external Serial Advanced Technology Attachment

USB - Universal Serial Bus

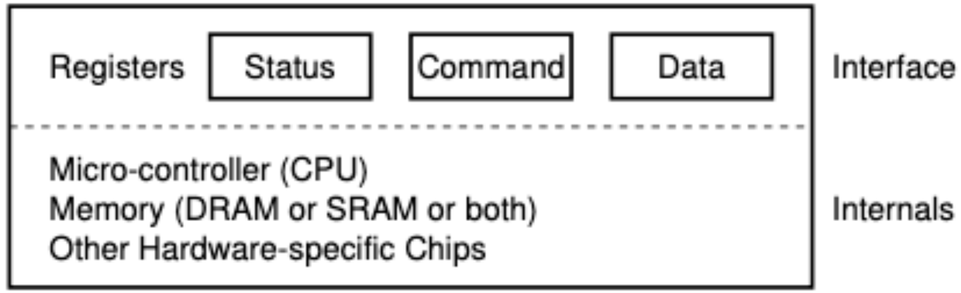
Canonical I/O protocol

```
while (STATUS == BUSY)

write data to DATA register

write command to COMMAND register

while (STATUS == BUSY)
```



4 steps of the canonical protocol

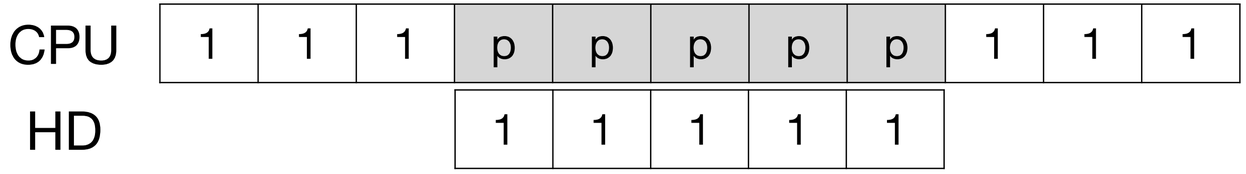
1. OS polls the device to see if it's available
2. OS sends data (e.g. smth to be written to memory)
3. Write command -> device assumes that the data was given too and starts
4. OS polls the device again to see if it's done

Canonical I/O protocol

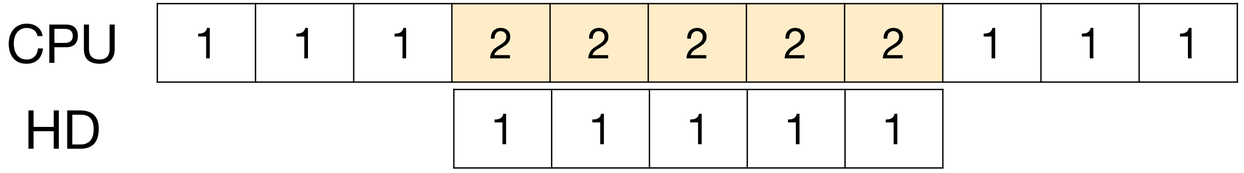
- Problems with canonical protocol
  - Standard interface is limited -> you cannot use the device fully
  - Requires drivers -> 70% of Linux kernel is drivers for I/O devices
  - Polling wastes CPU cycles

CPU overhead

- Polling wastes CPU cycles
- Interrupts allow better CPU and HD utilization
  - OS issues a request
  - puts process to sleep
  - switch to a different process
  - switch back when I/O device sends an interrupt



CPU repeatedly polls the device while process is performing I/O

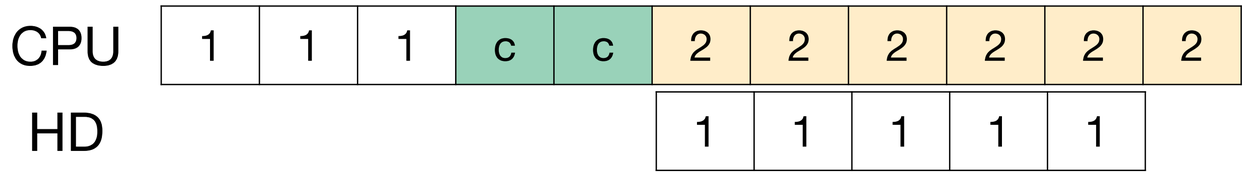


with interrupts, CPU can run something else while the I/O is being done

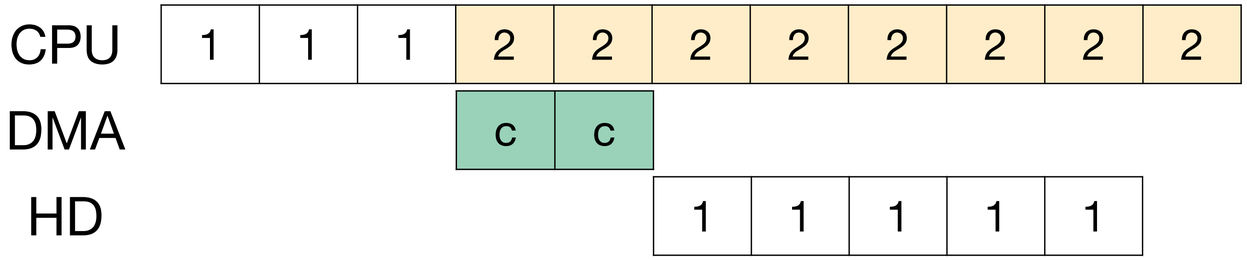
DMA

- Direct memory access (DMA) - rely on hardware for I/O
  - OS tells DMA
    - where the data is in RAM
    - what to copy
    - which device to send it to
  - DMA does the rest and raises an interrupt when done

data copying still uses CPU



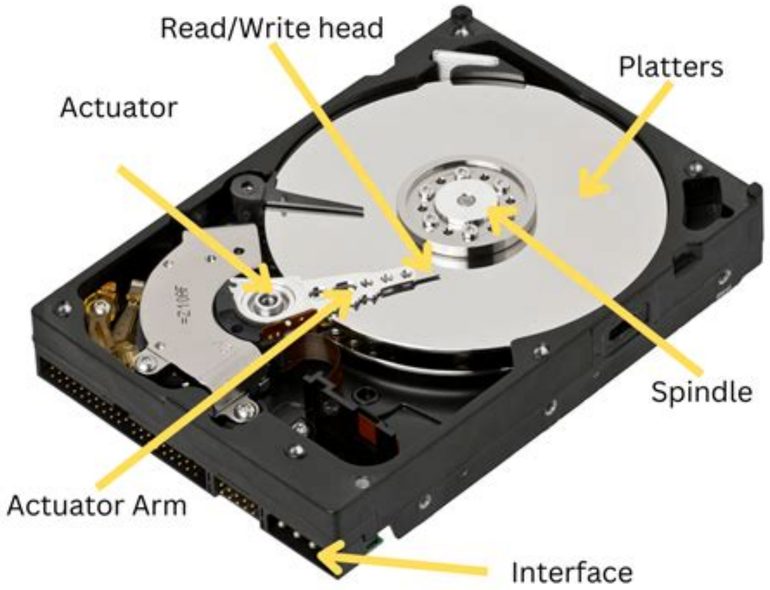
OS can delegate to DMA instead and use CPU for more useful work



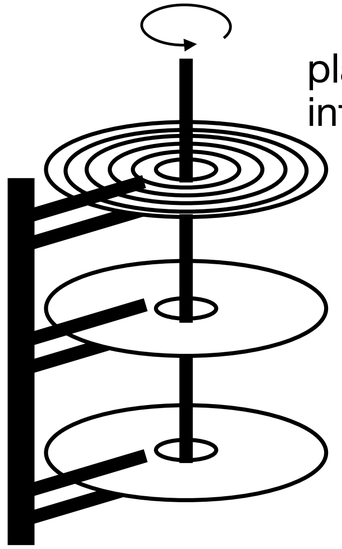
Hard drive

- The main form of persistent data storage since 1960s
  - lower cost, higher reliability, and higher storage capacity than SSDs
- Why do we study this in the OS course?
- HDD is a **mechanical** device
  - **Moving** physical parts takes time
  - Physical **layout** of the device affects how the data is stored
  - Both affect how file systems are designed

HD anatomy



platters rotate
around the spindle



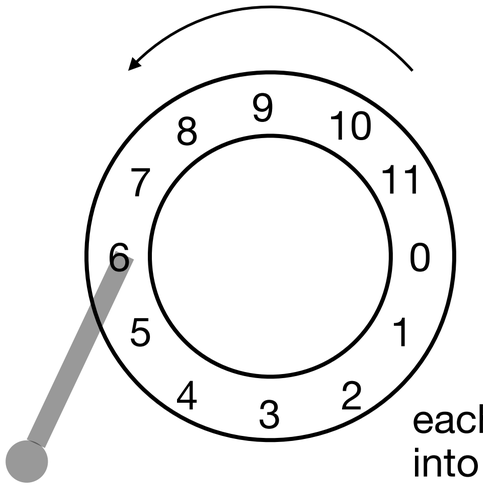
platters are split
into tracks

arm moves to read tracks

heads at the end of each arm read/
write onto platters on both sides

magnetizes a film of ferromagnetic
material coating the plates

platters rotate CCW

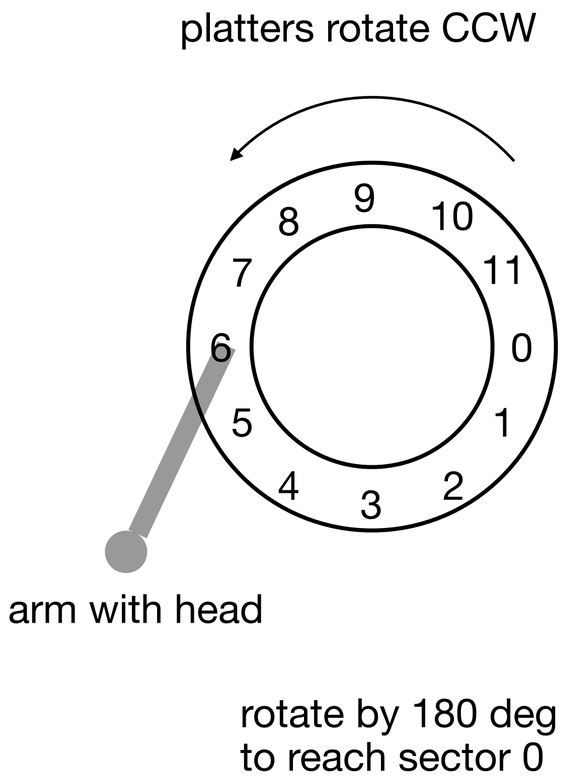


arm with head

each track is split
into sectors 512-
bytes each

HD latency

- Each platter track is divided into sectors
  - sequentially numbered
  - 512-bytes in size
- Arm is positioned above the platter
  - can read/write to any sector underneath it
  - platter rotates to give access to sectors



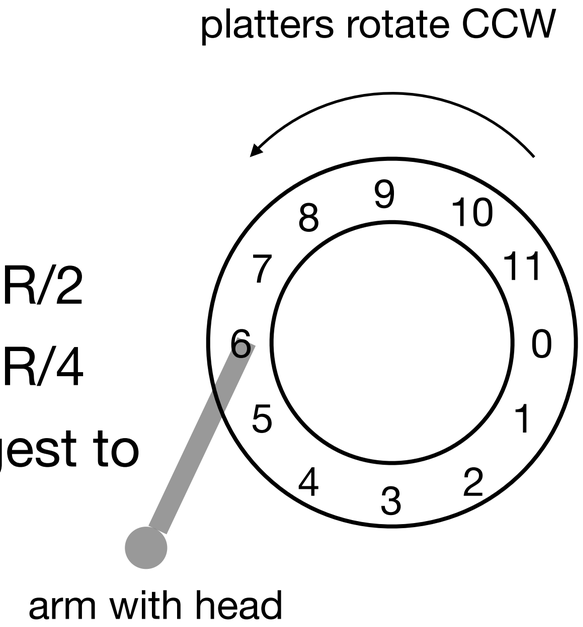
Single-track latency

- **Rotational delay** - R (in ms)
  - 360 deg rotation of the plate

accessing 0 from this position is R/2

accessing 9 from this position is R/4

which sector would take the longest to access from here?



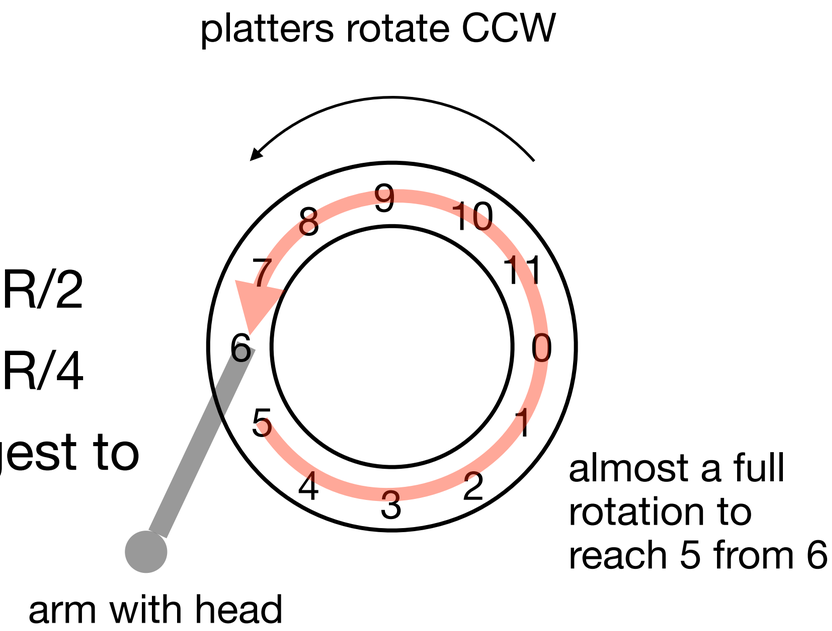
Single-track latency

- **Rotational delay** - R (in ms)
  - 360 deg rotation of the plate

accessing 0 from this position is R/2

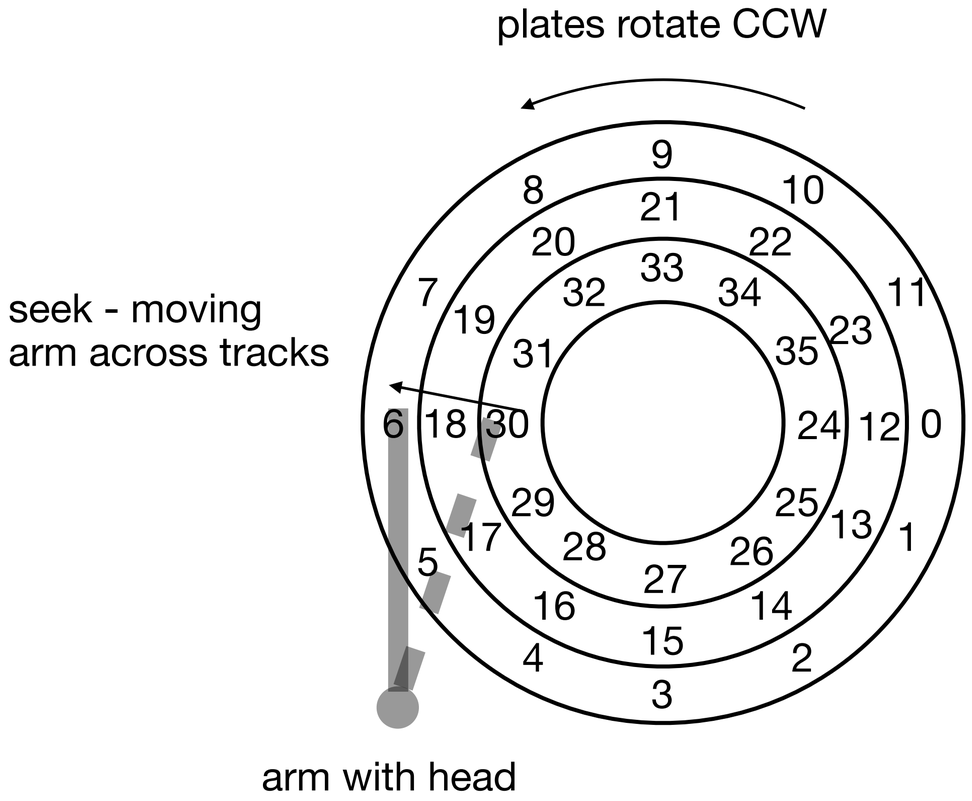
accessing 9 from this position is R/4

which sector would take the longest to access from here? **5**



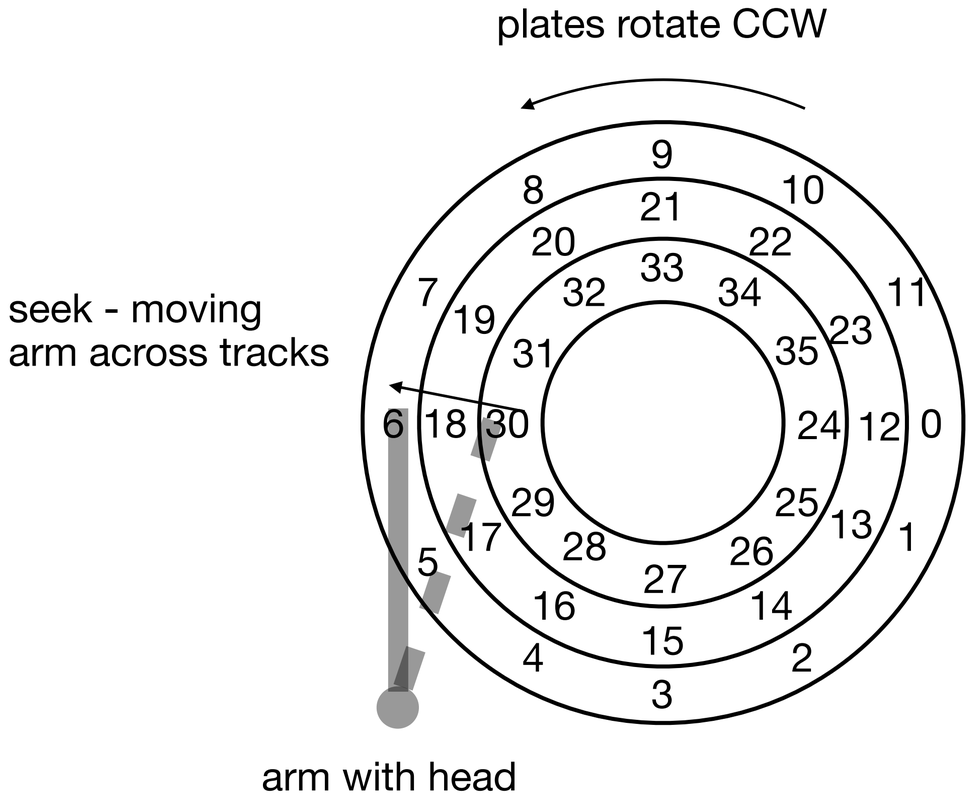
Multi-track latency

- **Rotational delay** - R (in ms)
  - 360 deg rotation of the plate
- **Seek time** - arm moving to the right track



Multi-track latency

- **Rotational delay** - R (in ms)
  - 360 deg rotation of the plate
- **Seek time** - arm moving to the right track
  - 4 phases each with delays
    - acceleration - arm starts to move
    - coasting - steady movement at full speed
    - deceleration - slowing down
    - settling - precise positioning above the track
- **Transfer time** - reading/writing data



I/O time

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

total time for I/O operation

time to move the arm across tracks

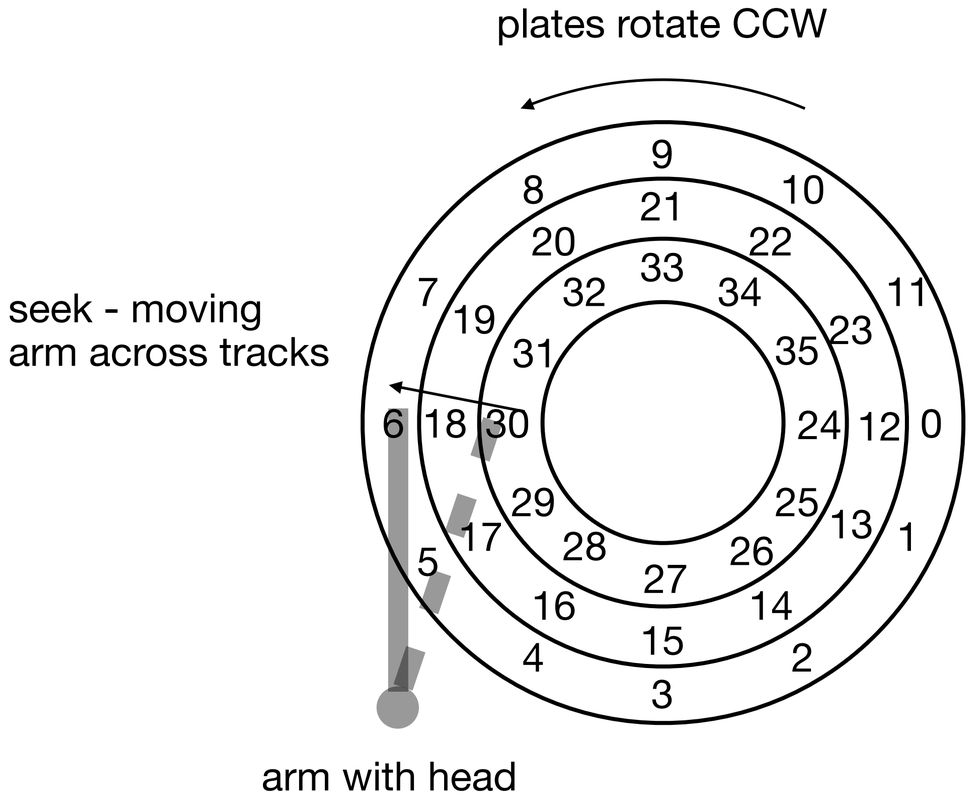
time to rotate the platters

time to write/ read data

$$R_{I/O} = \frac{\text{Size}_{Transfer}}{T_{I/O}}$$

I/O rate

amount of data divided by the total I/O time

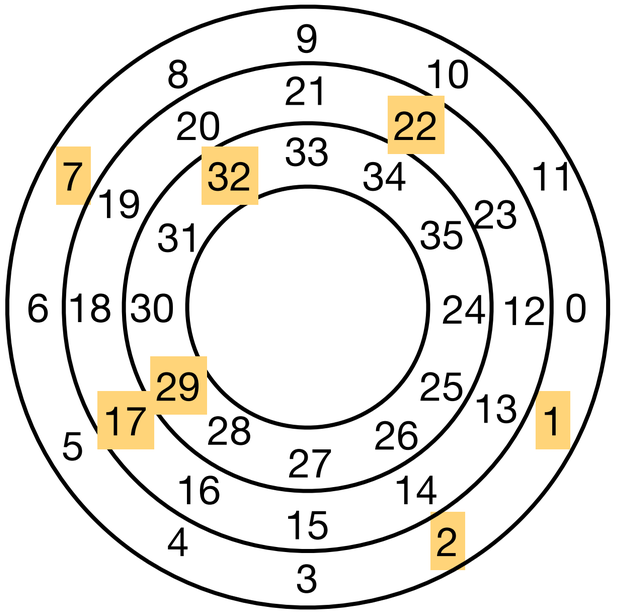


HD specs

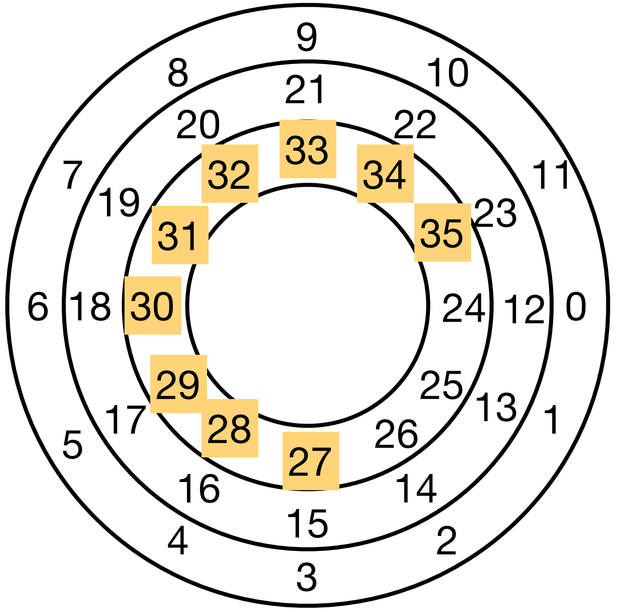
- **Capacity** - how much data can fit
- **RPM** - how fast the platters rotate
- **Avg. seek (ms)** - how fast the arm moves
- **Number of platters**
- **Cache** - usually in MB
- **Interface**
  - SCSI - 1980s standard for high-performance and transfer rates, used in high-end servers and workstations
  - SATA - 2000s standard for hard drives, more common

HD benchmarking

- Typical workloads
  - **Random**
    - issue multiple small (4KB) reads from random locations on the disk
    - tests average single-request latency
  - **Sequential**
    - issue multiple consecutive 4KB reads
    - tests steady-state throughput



random workload



sequential workload

HD benchmarking

- Typical workloads
  - **Random**
    - issue multiple small (4KB) reads from random locations on the disk
    - tests average single-request latency
  - **Sequential**
    - issue multiple consecutive 4KB reads
    - tests steady-state throughput

	Seagate ST600MP0005	**Seagate ST800DMZ04**
Capacity	600 GB	8TB
RPM	15,000	5,400
Avg. seek	2 ms	8.5 ms
Max transfer rate	2400 MB/s	190 MB/s
Platters	3	6
Cache	128 MB	256 MB
Interface	SCSI	SATA
$R_{I/O}$ Random	?	?
$R_{I/O}$ Sequential	?	?

need to compute I/O rate for both drives and both workloads

HD benchmarking

- Random workload SCSI drive
- T seek for random workload is avg from the specs - 2 ms
- T rotation - derive from RPM - 2ms

$$T_{rotation} = \frac{1s}{RPS} = \frac{1s}{RPM/60}$$

$$= \frac{1s}{15000/60} = 1s/250 = 0.004s = 4ms$$

will be 1/2 rotation on average, so $T_{rotation} = 2ms$

- T transfer - derive from MTR - 0.0001 ms

$$T_{transfer} = \frac{4KB \cdot 1s}{2400MB} = \frac{4KB \cdot 1s}{2400000KB} = 0.0001ms$$

	Seagate ST600MP0005	**Seagate ST800DMZ04**
Capacity	600 GB	8TB
RPM	15,000	5,400
Avg. seek	2 ms	8.5 ms
Max transfer rate	2400 MB/s	190 MB/s
Platters	3	6
Cache	128 MB	256 MB
Interface	SCSI	SATA
$R_{I/O}$ Random	10 MB/s	?
$R_{I/O}$ Sequential	?	?

need to compute I/O rate for both drives and both workloads

$$R_{I/O} = \frac{\text{Size}_{Transfer}}{T_{I/O}} = \frac{4KB}{4ms} = \frac{0.04MB}{0.004s} = 10MB/s$$

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

$$= 2ms + 2ms + 0.0001ms = 4.0001ms$$

HD benchmarking

- Random workload SATA drive
- T seek for random workload is avg from the specs - 8.5 ms
- T rotation - derive from RPM - 5 ms

$$T_{rotation} = \frac{1s}{RPS} = \frac{1s}{RPM/60}$$

$$= \frac{1s}{5400/60} = 1s/90 = 0.01 = 10ms$$

will be 1/2 rotation on average, so $T_{rotation} = 5ms$

- T transfer - derive from MTR - 0.005 ms

$$T_{transfer} = \frac{4KB \cdot 1s}{190MB} = \frac{4KB \cdot 1s}{190000KB} = 0.02ms$$

	Seagate ST600MP0005	**Seagate ST800DMZ04**
Capacity	600 GB	8TB
RPM	15,000	5,400
Avg. seek	2 ms	8.5 ms
Max transfer rate	2400 MB/s	190 MB/s
Platters	3	6
Cache	128 MB	256 MB
Interface	SCSI	SATA
$R_{I/O}$ Random	10 MB/s	3 MB/s
$R_{I/O}$ Sequential	?	?

need to compute I/O rate for both drives and both workloads

$$R_{I/O} = \frac{\text{Size}_{Transfer}}{T_{I/O}} = \frac{4KB}{4.005ms} = \frac{0.04MB}{0.0135} = 3MB/s$$

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

$$= 8.5ms + 5ms + 0.002ms = 13.502ms$$

HD benchmarking

- Sequential workload SCSI drive
- Assume 100 MB transfer
- T seek, T rotation - only one needed
- T transfer - derive from MTR - 0.13s

$$T_{transfer} = \frac{100MB \cdot 1s}{2400MB} = 0.042s$$

	Seagate ST600MP0005	**Seagate ST800DMZ04**
Capacity	600 GB	8TB
RPM	15,000	5,400
Avg. seek	2 ms	8.5 ms
Max transfer rate	2400 MB/s	190 MB/s
Platters	3	6
Cache	128 MB	256 MB
Interface	SCSI	SATA
$R_{I/O}$ Random	10 MB/s	3 MB/s
$R_{I/O}$ Sequential	2173 MB/s	?

need to compute I/O rate for both drives and both workloads

$$R_{I/O} = \frac{\text{Size}_{Transfer}}{T_{I/O}} = \frac{100MB}{0.046s} = 2173MB/s$$

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$
$$= 2 + 2 + 0.042s = 0.046s$$

HD benchmarking

- Sequential workload SATA drive
- Assume 100 MB transfer
- T seek, T rotation - only one needed
- T transfer - derive from MTR - 0.52 s

$$T_{transfer} = \frac{100MB \cdot 1s}{190MB} = 0.52s$$

	Seagate ST600MP0005	**Seagate ST800DMZ04**
Capacity	600 GB	8TB
RPM	15,000	5,400
Avg. seek	2 ms	8.5 ms
Max transfer rate	2400 MB/s	190 MB/s
Platters	3	6
Cache	128 MB	256 MB
Interface	SCSI	SATA
$R_{I/O}$ Random	10 MB/s	3 MB/s
$R_{I/O}$ Sequential	2173 MB/s	188 MB/s

need to compute I/O rate for both drives and both workloads

$$R_{I/O} = \frac{\text{Size}_{Transfer}}{T_{I/O}} = \frac{100MB}{0.52s} = 188MB/s$$

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$
$$= 8.5ms + 5ms + 0.52s = 0.53s$$

HD benchmarking

- Random I/O rate is lower than sequential
  - 217x lower for SCSI
  - 64x lower for SATA
- SCSI faster than SATA
  - 3x on random
  - 11x on sequential

	Seagate ST600MP0005	**Seagate ST800DMZ04**
Capacity	600 GB	8TB
RPM	15,000	5,400
Avg. seek	2 ms	8.5 ms
Max transfer rate	2400 MB/s	190 MB/s
Platters	3	6
Cache	128 MB	256 MB
Interface	SCSI	SATA
$R_{I/O}$ Random	10 MB/s	3 MB/s
$R_{I/O}$ Sequential	2173 MB/s	188 MB/s

Disk scheduling

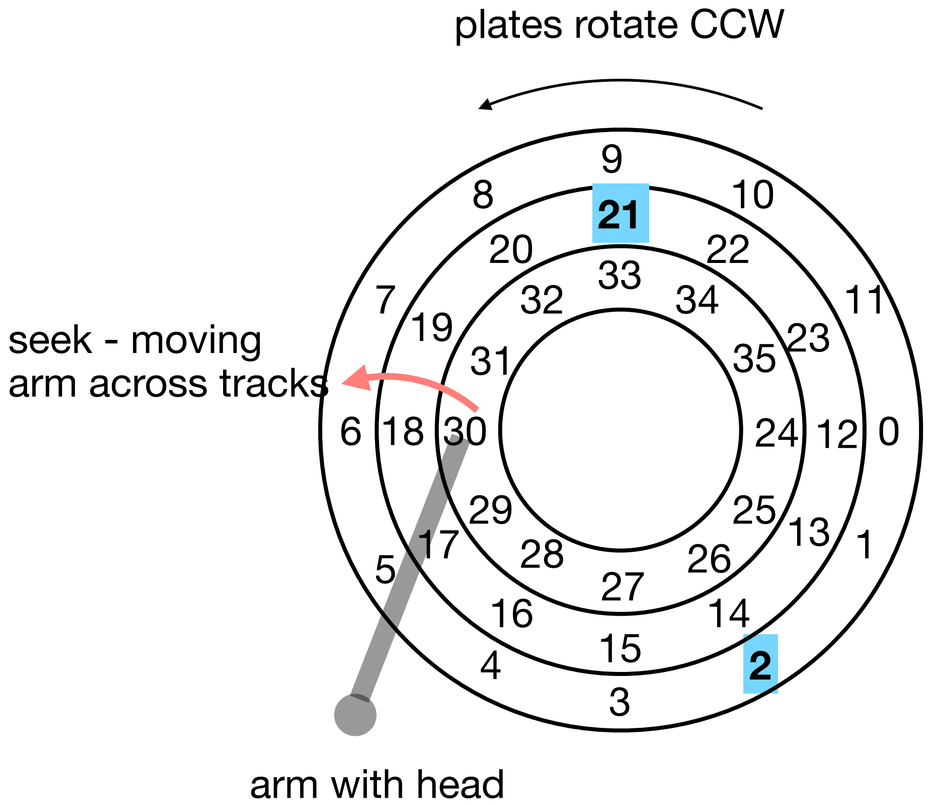
- I/O operations are scheduled too!
  - I/O operations are slow -> order matters
- Differences from CPU scheduling
  - Process duration is rarely known in advance
  - I/O duration can be inferred
    - how much data is transferred
    - where it will be written
    - device parameters

Disk scheduling

- Differences from CPU scheduling
  - Process duration is rarely known in advance
  - I/O duration can be inferred
    - how much data is transferred
    - where it will be written
    - device parameters
  - **If all jobs arrive at the same time and duration is known known**
    - what is the best scheduling approach for the highest turnaround?
    - SJF! Many I/O schedulers approximate it

SSTF

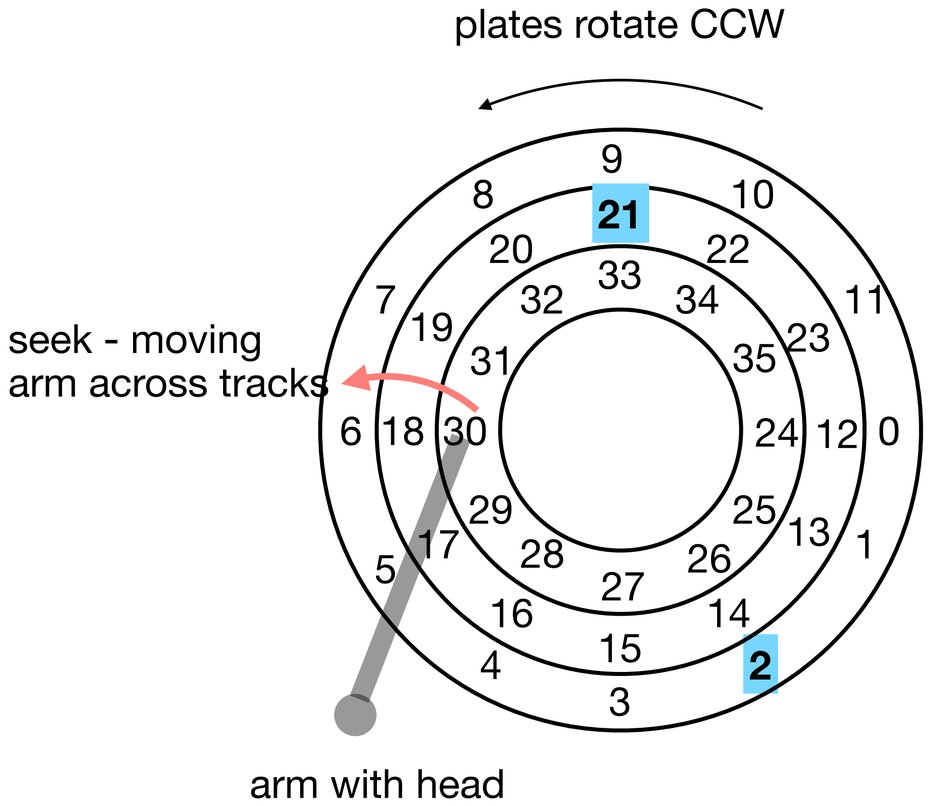
- Option #1 SSTF - Shortest seek time first
  - Order requests by track
  - Pick the nearest track to complete first



arm is at sector 30 (inner track)
requests arrive for 21 and 2
which one should be served next?

SSTF

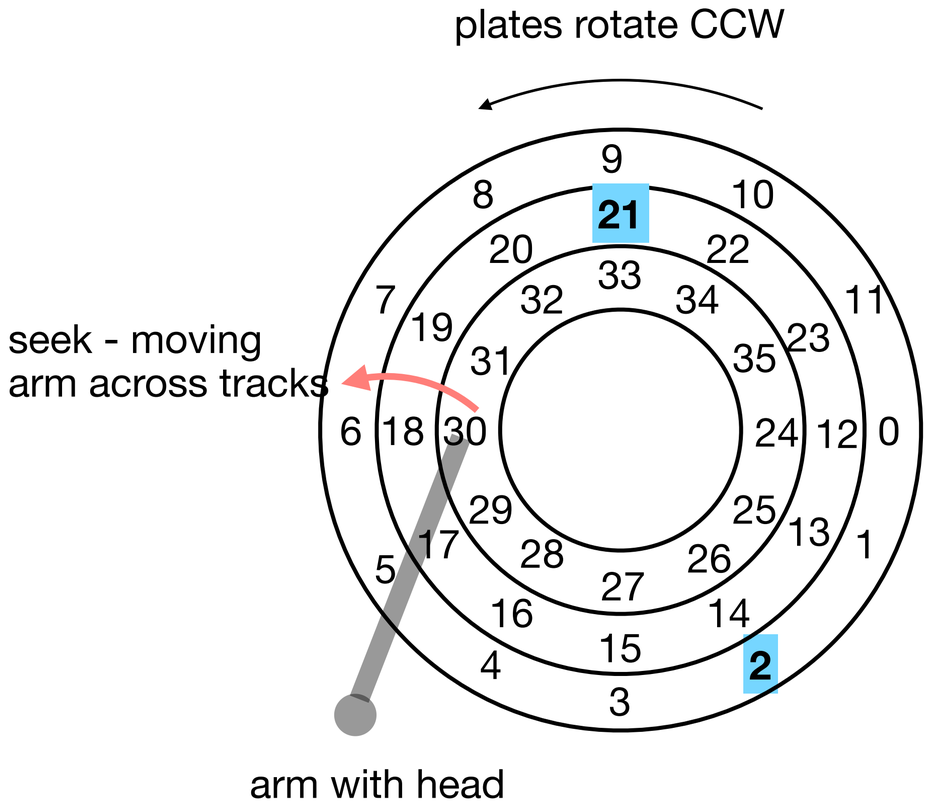
- Option #1 SSTF - Shortest seek time first
  - Order requests by track
  - Pick the nearest track to complete first



arm is at sector 30 (inner track)
requests arrive for 21 and 2
which one should be served next?
21 (middle) then 2 (outer)

SSTF

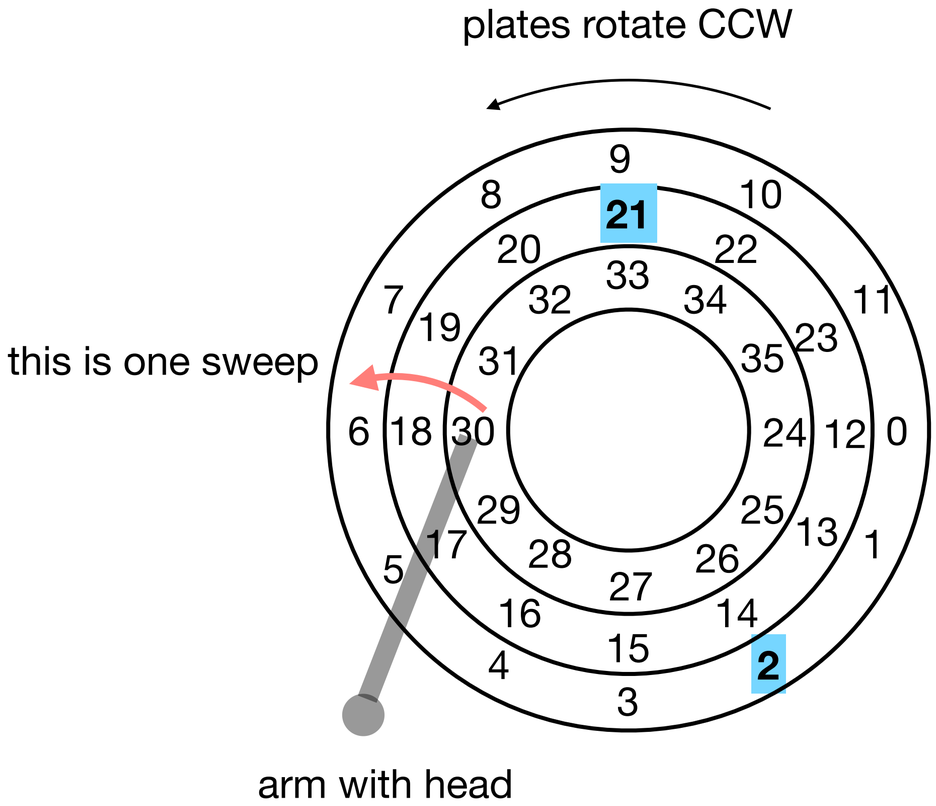
- Option #1 SSTF - Shortest seek time first
  - Order requests by track
- Cons:
  - starvation is possible
    - e.g. in our example inner track requests can starve requests to other tracks



arm is at sector 30 (inner track)
requests arrive for 21 and 2
which one should be served next?
21 (middle) then 2 (outer)

SCAN

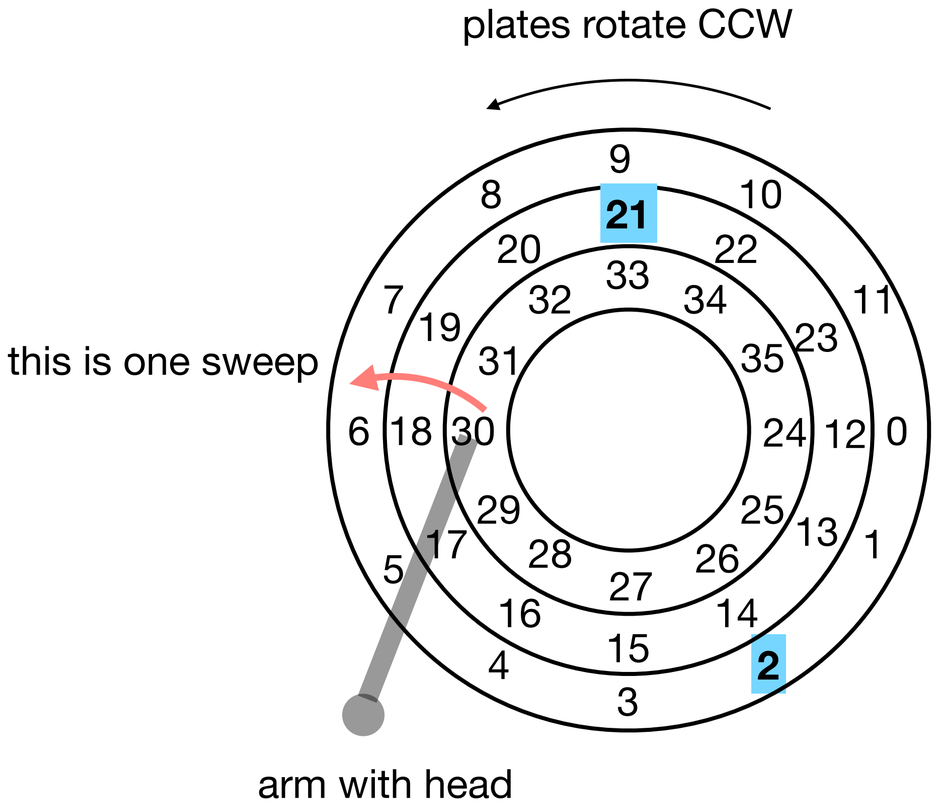
- Option # 2 - Move arm back and forth across the disk
- Sweep — one pass
  - from inner to outer or from outer to inner
- Service requests in the order of the sweep
- SCAN is also called the elevator algorithm
  - like an elevator going up and down



arm is at sector 30 (inner track)
requests arrive for 21 and 2
which one should be served next?

SCAN

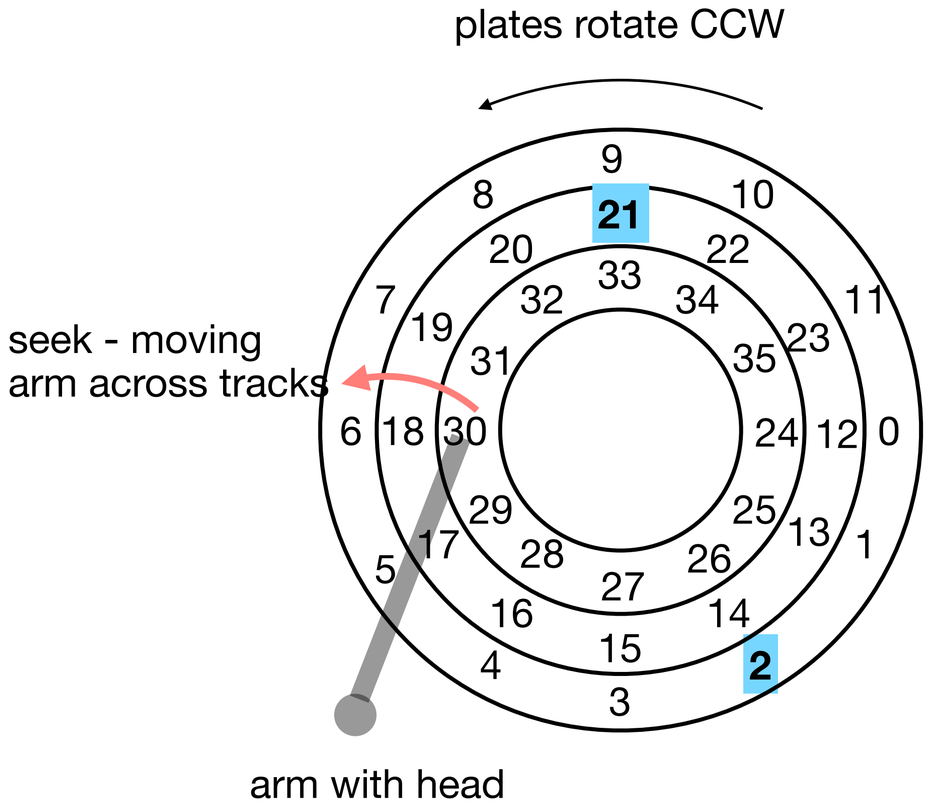
- Move arm back and forth across the disk
- Sweep — one pass
  - from inner to outer or from outer to inner
- Service requests in the order of the sweep



arm is at sector 30 (inner track)
requests arrive for 21 and 2
which one should be served next?
21 (middle) then 2 (outer)

SCAN

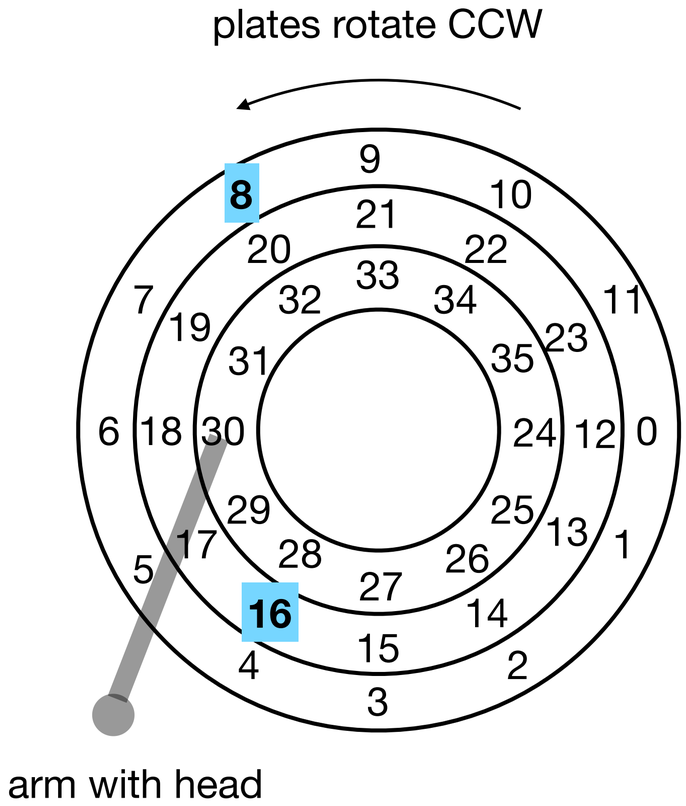
- Move arm back and forth across the disk
- Sweep — one pass
  - from inner to outer or from outer to inner
- Service requests in the order of the sweep
- Problem #1: **starvation possible** if requests keep coming during the sweep
  - F-SCAN - freeze the queue during the sweep
    - schedule all new arrivals later
- Problem #2: back and forth sweeps favour the **middle tracks**
  - C-SCAN - sweep in one direction and reset



arm is at sector 30 (inner track)
requests arrive for 21 and 2
which one should be served next?
21 (middle) then 2 (outer)

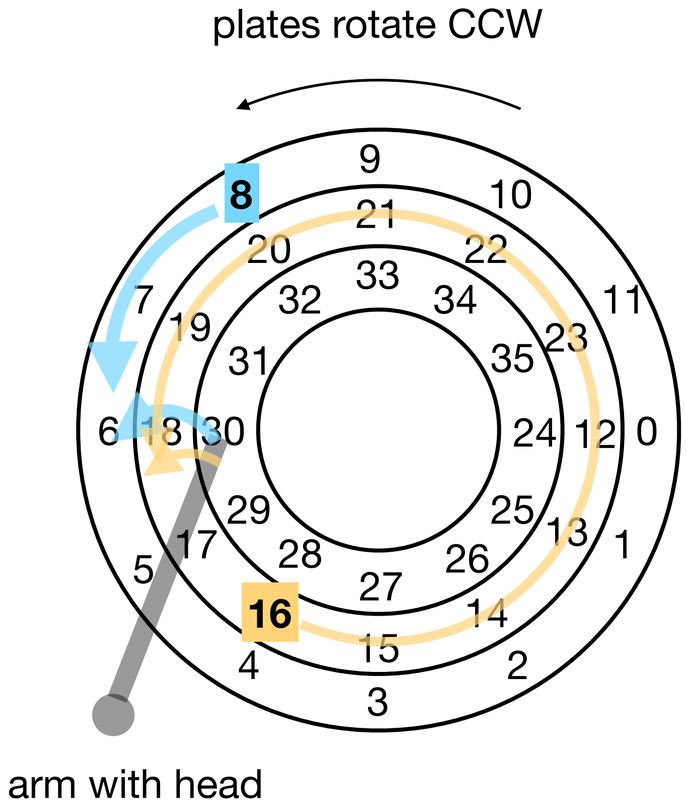
SPTF

- SSTF and SCAN **ignore rotation**
- Option # 3 - Shortest Positioning Time First (**SPTF**) to the rescue!
- Positioning time is seek + rotation
  - Depends on
    - device's avg seek and RPM
    - request location



SPTF

- Positioning time is seek + rotation
- Depends on
  - device's avg seek and RPM
  - request



arm is at sector 30 (inner track)
requests arrive for 8 and 16
which one should be served next?

if seek > rotation? 16 then 8
if seek < rotation? 8 then 16

Disk scheduling

- Option #1 **SSTF** - Shortest seek time first
  - order requests by track (block). Cons: starvation
- Option #2 **SCAN** - sweep tracks back and forth servicing requests
  - Cons: starvation, favouring middle tracks -> F-SCAN and C-SCAN variants
- Option #3 **SPTF** - take seek+rotation into account
- More options
  - OS can **delegate** scheduling to the hard drive itself
  - **Merge requests** - grab two neighbouring sectors together

Summary so far

- A hierarchy of I/O devices
  - expensive, high-bandwidth, low-capacity devices are closer to CPU
    - RAM, GPU, etc.
  - cheap, lower-bandwidth, high-capacity devices are farther away from CPU
    - SATA drives, USB devices, etc.
- Properties of hard drives affect how data is read/written
  - seeking and transferring data has latency
- I/O jobs can be scheduled to reduce latency
  - SSTF - shortest seek time
  - SCAN - sweeping the tracks back and forth
  - SPTF - minimize seek+rotation time

RAIDs

- Redundant Array of Inexpensive Disks - **RAIDs**
  - cheap - built out of **consumer-grade** hard drives
  - **large capacity**
  - **reliable**
  - **transparent** - system sees it as a single large HD
- Often a standalone device with
  - standard connection (SCSI or SATA)
  - processor, memory and cache
  - specialized software to manage disks

RAIDs

- Evaluation criteria
  - **Capacity** - how much disk space is available
  - **Reliability** - how many disks faults can be tolerated?
  - **Performance** on typical workloads
    - Random - tests single-request latency
    - Sequential - steady-state throughput

RAIDs

- **Fault-stop model**
  - detects failed drives and prevents writing to them
- Different RAID designs
  - RAID Level 0 - **striping**
  - RAID Level 1 - **mirroring**
  - RAID level 4/5 - **parity**-based redundancy

RAID Level 0

- Uses striping - distributing blocks of the array across disks in a round-robin fashion
  - Better parallelism
- No redundancy
- Serves as upper-bound on performance and capacity

This is how RAID appears to the OS

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

this is one block

Blocks are grouped into chunks and written to different drives

stripe - blocks in the same row

Disk 0	Disk 1	Disk 2	Disk 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

This is how the data is stored across hard drives in RAID Level 0

RAID Level 0

- RAID mapping
  - translate logical to physical block
- `disk = logical block % num disks`
- `offset = logical block / num disks`

This is how RAID appears to the OS

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

logical block

Translate logical block 14

disk = 14 % 4 = 2

offset = 14 / 4 = 3

Blocks are grouped into chunks and written to different drives

stripe - blocks in the same row

Disk 0	Disk 1	Disk 2	Disk 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

This is how the data is stored across hard drives in RAID Level 0

RAID Level 0

- Chunk size affects performance
  - too small - many files will be striped across disks
    - better parallelism for reads and writes to a single file
    - positioning time will increase - max across all drives
  - too large - worse for parallelism but better for positioning

Each chunk can have 1 or more blocks

Disk 0	Disk 1	Disk 2	Disk 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

chunk with 1 block

chunk with 2 blocks

Disk 0	Disk 1	Disk 2	Disk 3
0	2	4	6
1	3	7	7
8	10	14	14
9	13	15	16

This is how the data is stored across hard drives in RAID Level 0

RAID Level 0

- **Capacity** - OK
  - N disks with B blocks -> N*B blocks of useful capacity
- **Reliability** - not OK
  - cannot tolerate disk failures -> data loss
- **Performance** - OK
  - high parallelism, uses all disks to service I/O requests

Workloads

- Measure transfer rates for two workloads
  - **Random** - tests single-request latency
    - issue multiple small requests to random locations
    - more time spent seeking and less time transferring
  - **Sequential** - tests steady-state throughput
    - issue sequential requests for a large contiguous chunk of memory
    - less time spent seeking and more time transferring

Workloads

- Avg seek time - 7ms
- Avg rot. delay - 3 ms
- Transfer rate of disk - 50 MB/s
- Sequential transfer avg 10 MB
- Random transfer avg 10 KB

Rate of sequential transfer

$$S = \frac{\text{Amount of data}}{T_{I/O}} = \frac{10MB}{210ms} = 47.62MB/s$$

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

$$= 7 + 3 + \frac{10MB}{50MB/s} = 7 + 3 + 200ms = 210ms$$

data transfer dominates

Rate of random transfer

$$R = \frac{\text{Amount of data}}{T_{I/O}} = \frac{10KB}{10.195ms} = 0.981MB/s$$

$$T_{I/O} = 7 + 3 + \frac{10KB}{50MB/s} = 7 + 3 + 0.195ms = 0.981ms$$

seek time dominates

RAID Level 0

N - number of disks
B - number of blocks in each disk

R - random throughput
S - sequential throughput

- **Capacity** - OK
  - N disks with B blocks -> N*B blocks of useful capacity
- **Reliability** - not OK
  - cannot tolerate disk failures -> data loss
- **Performance** - OK
  - high parallelism, uses all disks to service I/O requests
- **Sequential workload** - OK
  - N*S MB/s -> num disks multiplied by sequential throughput
- **Random workload** - OK
  - N*R MB/s -> num disks multiplied by random throughput

this is an upper bound on RAID performance

cannot get any better than this

RAID Level 1

- **Mirroring** - make copies of each block on different disks
- Better disk **failure tolerance**
- Reading from RAID-1
  - choose any block
- Writing to RAID-1
  - write to both blocks in parallel

Mirroring in RAID 1
Each block is duplicated on a different drive

Disk 0	Disk 1	Disk 2	Disk 3
0	0	1	1
2	2	3	3
4	4	5	5
6	6	7	7

RAID-10 stripe of mirrors

Disk 0	Disk 1	Disk 2	Disk 3
0	1	0	1
2	3	2	3
4	5	4	5
6	7	6	7

RAID-10 mirror of stripes

RAID Level 1

N - number of disks
B - number of blocks in each disk

R - random throughput
S - sequential throughput

- **Capacity** - not OK
  - With mirroring level = 2 only half of useful capacity
  - (N*B)/2
- **Reliability** - OK
  - can tolerate 1 disk failure and up to N/2 failures
- **Performance** - slower than RAID-0
  - read is same as RAID-0
  - writing is slower (+ potential inconsistencies due to timing)
- **Sequential workload** - worse than RAID-0
  - N/2*S MB/s -> half the max bandwidth for both reading and writing
- **Random workload** - same as RAID-0 for reading, half for writing
  - N*R MB/s for reading -> can distribute reads across all disks
  - N/2*R MB/s for writing -> each logical write requires 2 physical writes

RAID Level 4

- **Parity** - add a parity block for each stripe of data
  - stores redundant data
  - reduces space penalty of mirroring

RAID-4 with parity

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

add a block of data for each stripe on a separate drive

only works if 1 value is lost!

restoring data using parity

if one of the values is lost

to get back the lost value compute XOR of remaining data and parity bit -> XOR(0,0,1,0)=1

				P
0	0	~~1~~	1	0
0	1	0	0	1

Tip: XOR is 0 when there are an even number of 1s in the input and 1 otherwise

parity is computed for each bit in the stripe and stored in the parity block

				P
0	0	1	1	XOR(0,0,1,1)=0
0	1	0	0	XOR(0,1,0,0)=1

RAID Level 4

N - number of disks
B - number of blocks in each disk

R - random throughput
S - sequential throughput

- **Capacity** - better than RAID-1
  - (N-1)*B - lost one drive to storing parity blocks
- **Reliability** - worse than RAID-1
  - can tolerate **only 1 disk failure**

RAID Level 4

- **Writing to RAID-4**
- If we modify one block in the stripe -> need to update the parity
- **Option #1 Additive parity**
  - recompute the parity bit

RAID Level 4

- **Writing to RAID-4**
- If we modify one block in the stripe -> need to update the parity
- **Option #1 Additive parity**
  - recompute the parity bit
- **Option #2 Subtractive parity**
  - if new val is the same -> keep the parity bit
  - if new val is different -> flip the parity bit

				P
0	~~0~~ 1	1	1	XOR(0,0,1,1)=~~0~~ 1
0	1	0	0	XOR(0,1,0,0)=1

RAID Level 4

- Sequential workload
  - reads will require (N-1)*S MB/s
    - one of the disks is used for parity
  - writes will require (N-1)*S MB/s
    - parity disk is used but does not increase throughput

RAID-4 with parity

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

full-stripe write - all blocks in a stripe can be written in parallel to different drives + parity bit to a separate drive

N - number of disks

B - number of blocks in each disk

R - random throughput

S - sequential throughput

RAID Level 4

- Random workload
  - reads will require (N-1)*S MB/s
    - same as before
    - random reads will be spread out but one of the disks is used for parity
  - **writes will require R/2 MB/s**
    - parity disk will need to perform 1 reads and 1 writes per logical write

RAID-4 with parity

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0*
4	5	6	7	P1
8	9	10	11	P2*
12	13	14	15	P3

small-write problem - if need to update 1 or more values, parity disk will be a bottleneck

N - number of disks

B - number of blocks in each disk

R - random throughput

S - sequential throughput

RAID Level 4

N - number of disks
B - number of blocks in each disk

R - random throughput
S - sequential throughput

- **Capacity** - better than RAID-1
  - (N-1)*B - lost one drive to storing parity blocks
- **Reliability** - worse than RAID-1
  - can tolerate only 1 disk failure
- **Performance** - worse than RAID-0
  - read is same as RAID-0
  - write 2x of RAID-0 (with subtractive parity needs 2 reads and 2 writes)
- **Sequential workload** - worse than RAID-0
  - N/2*S MB/s -> half the max bandwidth for both reading and writing
- **Random workload** - same as RAID-0 for reading, half for writing
  - N*R MB/s for reading -> can distribute reads across all disks
  - R/2 MB/s for writing -> each logical write requires 2 ops on parity drive

RAID Level 5

- Rotating parity - rotate parity block across drives
  - removes the parity drive bottleneck

RAID-4 with parity

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

RAID-5 with rotating parity

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
5	6	7	P1	4
10	11	P2	8	9
15	P3	12	13	14
P4	16	17	18	19

distribute parity blocks across drives to remove the bottleneck

RAID Level 5

N - number of disks
B - number of blocks in each disk

R - random throughput
S - sequential throughput

- **Capacity** - same as RAID-4
- **Reliability** - same as RAID-4
- **Performance** - same as RAID-4
- **Sequential workload** - same as RAID-4
- **Random workload** - better than RAID-4
  - random read - N*R MB/s - can use all disks because parity blocks are distributed
  - random write - N/4*R MB/s - need 4 I/O operations for each read as but can use all drives

RAID summary

	RAID-0	RAID-1	RAID-4	RAID-5
Capacity	$N \cdot B$	$(N \cdot B)/2$	$(N-1) \cdot B$	$(N-1) \cdot B$
Reliability	0	1 (for sure) $\frac{N}{2}$ (if lucky)	1	1
Throughput				
Sequential Read	$N \cdot S$	$(N/2) \cdot S^1$	$(N-1) \cdot S$	$(N-1) \cdot S$
Sequential Write	$N \cdot S$	$(N/2) \cdot S^1$	$(N-1) \cdot S$	$(N-1) \cdot S$
Random Read	$N \cdot R$	$N \cdot R$	$(N-1) \cdot R$	$N \cdot R$
Random Write	$N \cdot R$	$(N/2) \cdot R$	$\frac{1}{2} \cdot R$	$\frac{N}{4} R$
Latency				
Read	T	T	T	T
Write	T	T	$2T$	$2T$

N - number of disks

B - number of blocks in each disk

R - random throughput

S - sequential throughput

Summary

- Hard drive - seek, rotational, and transfer latency
- HD benchmarking - random and sequential workloads
- I/O scheduling - prioritize short I/O jobs first
  - SSTF (fastest seek), SCAN (sweep), SPTF (seek+rot)
- RAIDs - Redundant Arrays of Inexpensive Disks
  - RAID-0 - no redundancy, upper bound on performance
  - RAID-1 - striping - distributing data across drives for performance
  - RAID -2 - mirroring - duplicating data for redundancy, worse performance
  - RAID-4/5 - parity - balances redundancy with performance

Reading

- Chapter 36 I/O devices
  - https://pages.cs.wisc.edu/~remzi/OSTEP/file-devices.pdf
- Chapter 37 HDD
  - https://pages.cs.wisc.edu/~remzi/OSTEP/file-disks.pdf
- Chapter 38 RAIDs
  - https://pages.cs.wisc.edu/~remzi/OSTEP/file-raid.pdf
- Code who am I kidding with this
  - https://github.com/remzi-arpacidusseau/ostep-homework/tree/master/file-raid
  - https://github.com/remzi-arpacidusseau/ostep-homework/tree/master/file-devices
  - https://github.com/remzi-arpacidusseau/ostep-homework/tree/master/file-disks

Quiz 17

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999